Sprint 12 Structured Notes: Creating, Editing, and Removing DOM Content

0. What Sprint 12 Is About

Sprint 12 is about changing the web page with JavaScript.

In Sprint 11, you learned how JavaScript can **find** HTML elements on the page.

Example:

```
const title = document.getElementById("title");
```

That means:

```
Find the element with the id "title".
```

Sprint 12 teaches what to do **after** you find the element.

Example:

```
title.textContent = "New title";
```

That means:

```
Change the text inside the element.
```

The main goal of Sprint 12 is:

```
Change page content safely and intentionally.
```

By the end of Sprint 12, you should understand:

- how to change text on the page
- how to safely display plain text
- why `innerHTML` can be risky
- how to create new elements
- how to add elements to the page

- how to remove elements from the page
- how to add or change attributes
- how to use `classList`
- how to change inline styles
- why CSS classes are usually cleaner than direct style changes
- how to avoid unsafe DOM updates

Part A: Current Progress Before Sprint 12

1. What You Already Know

Before Sprint 12, you learned the foundation of JavaScript.

You learned:

Earlier Sprint	What You Learned	How It Helps in Sprint 12
Sprint 1	JavaScript can make a page respond to actions.	Sprint 12 changes the page after JavaScript finds elements.
Sprint 2	Values, variables, `let`, `const`, and types.	DOM elements are stored in variables.
Sprint 3	Strings and text processing.	Page text is often changed using strings.
Sprint 4	Conditions and booleans.	You can check whether an element exists before changing it.
Sprint 5	Functions, callbacks, and scope.	Later, DOM updates often happen inside functions.
Sprint 6	Arrays.	You may create many elements from array data.
Sprint 7	Loops.	You can loop through data and create repeated page elements.
Sprint 8	Objects.	Page data is often stored as objects before being displayed.
Sprint 9	Array methods and callbacks.	You may use array methods to prepare data before showing it.
Sprint 10	Sets and Maps.	You now understand different collection tools.
Sprint 11	Browser APIs, DOM, and element selection.	Sprint 12 builds directly on selecting page elements.

Simple summary:

```
Sprint 11 taught you how to find page elements.
Sprint 12 teaches you how to change page elements.
```

Part B: The Main Idea of Sprint 12

2. The DOM Update Process

A beginner DOM update usually follows this pattern:

```
1. HTML exists on the page.
2. JavaScript selects an element.
3. JavaScript changes that element.
```

Example HTML:

```
<h1 id="title">Old Title</h1>
```

Example JavaScript:

```
const title = document.getElementById("title");
title.textContent = "New Title";
```

Step-by-step:

```
1. The HTML page has a heading.
2. JavaScript finds the heading by its id.
3. JavaScript changes the heading text.
```

Result:

```
<h1 id="title">New Title</h1>
```

Mental model:

```
HTML element exists
↓
JavaScript selects it
```

```
↓
JavaScript changes text, attributes, classes, styles, or children
```

Part C: Changing Text with `textContent`

3. What Is `textContent`?

`textContent` is a property that lets JavaScript read or change the plain text inside an element.

Example HTML:

```
<h1 id="title">Old Title</h1>
```

Example JavaScript:

```
const title = document.getElementById("title");
title.textContent = "New Title";
```

The page changes from:

```
<h1 id="title">Old Title</h1>
```

to:

```
<h1 id="title">New Title</h1>
```

Simple meaning:

```
textContent changes the text inside an element.
```

4. Reading Text with `textContent`

You can also use `textContent` to read the text inside an element.

HTML:

```
<p id="message">Hello</p>
```

JavaScript:

```
const message = document.getElementById("message");
console.log(message.textContent);
```

Expected output:

```
Hello
```

5. Changing Text with `textContent`

HTML:

```
<p id="message">Loading...</p>
```

JavaScript:

```
const message = document.getElementById("message");
message.textContent = "Finished loading.";
```

Result:

```
<p id="message">Finished loading.</p>
```

6. Why `textContent` Is Safe for Plain Text

`textContent` treats the value as text.

Example:

```
const box = document.getElementById("box");
box.textContent = "<strong>Hello</strong>";
```

The browser displays the text:

```
<strong>Hello</strong>
```

It does not turn it into a bold element.

This matters because user input should usually be treated as plain text, not HTML.

Beginner rule:

```
Use textContent when you want to display normal text.
Use textContent when the text comes from a user.
```

Part D: `innerHTML`

7. What Is `innerHTML`?

`innerHTML` lets JavaScript read or change the HTML inside an element.

Example:

```
<div id="box"></div>
```

```
const box = document.getElementById("box");
box.innerHTML = "<strong>Hello</strong>";
```

Result:

```
<div id="box"><strong>Hello</strong></div>
```

The browser reads `"<strong>Hello</strong>"` as HTML.

So the word `Hello` appears bold.

Simple meaning:

```
innerHTML can insert HTML markup into the page.
```

8. `textContent` vs `innerHTML`

These two properties are different.

Example:

```
box.textContent = "<strong>Hello</strong>";
```

This displays the characters as plain text:

```
<strong>Hello</strong>
```

But this:

```
box.innerHTML = "<strong>Hello</strong>";
```

creates a real HTML element:

```
<strong>Hello</strong>
```

Comparison:

Property	What It Does	Safer for User Input?
`textContent`	Inserts plain text	Yes
`innerHTML`	Inserts HTML markup	No, not with user input

9. Why `innerHTML` Can Be Risky

`innerHTML` is powerful, but it can be dangerous when the value comes from a user.

Risky example:

```
box.innerHTML = userInput;
```

Why is this risky?

Because `innerHTML` tells the browser:

```
Treat this string as HTML.
```

If user input contains unsafe HTML-like content, the browser may treat it as part of the page structure instead of harmless text.

This kind of security problem is commonly connected to XSS, which means cross-site scripting.

Beginner meaning of XSS:

```
XSS happens when unsafe user-controlled content is inserted into a web page as
executable or active page content.
```

You do not need to master all browser security yet, but you do need this rule:

```
Do not use innerHTML with user input.
```

Safer version:

```
box.textContent = userInput;
```

Riskier version:

```
box.innerHTML = userInput;
```

10. When Is `innerHTML` Acceptable?

`innerHTML` can be acceptable when:

- the HTML string is fully controlled by you
- the value does not come from a user
- the value does not come from an untrusted source
- you intentionally want to create markup

Example:

```
box.innerHTML = "<strong>Saved successfully</strong>";
```

This is controlled by the developer.

But as a beginner, prefer this safer habit:

```
Use createElement() and textContent instead of innerHTML when building dynamic content.
```

Part E: Creating Elements with `createElement()`

11. What Is `createElement()`?

`createElement()` creates a new HTML element using JavaScript.

Example:

```
const item = document.createElement("li");
```

This creates a new list item element:

```
<li></li>
```

But it is not visible on the page yet.

Simple meaning:

```
createElement() creates an element in JavaScript memory.
```

12. `createElement()` Does Not Automatically Show the Element

This code creates an element:

```
const item = document.createElement("li");
```

But the page does not change yet.

Why?

Because the element has been created, but it has not been added to the DOM tree.

Beginner mental model:

```
createElement() = make the element
appendChild() = place the element on the page
```

13. Creating Different Elements

You can create many kinds of elements:

```
const paragraph = document.createElement("p");
const button = document.createElement("button");
const listItem = document.createElement("li");
const image = document.createElement("img");
const section = document.createElement("section");
```

Each one creates a new element object.

Examples:

```
const paragraph = document.createElement("p");
paragraph.textContent = "This is a paragraph.";
```

```
const button = document.createElement("button");
button.textContent = "Click me";
```

```
const item = document.createElement("li");
item.textContent = "Ice Cream";
```

Part F: Adding Elements with `appendChild()`

14. What Is `appendChild()`?

`appendChild()` adds a child element inside a parent element.

Example HTML:

```
<ul id="desserts"></ul>
```

JavaScript:

```
const list = document.getElementById("desserts");

const item = document.createElement("li");
item.textContent = "Ice Cream";

list.appendChild(item);
```

Result:

```
<ul id="desserts">
  <li>Ice Cream</li>
</ul>
```

Simple meaning:

```
appendChild() puts one element inside another element.
```

15. Parent and Child Review

In this result:

```
<ul id="desserts">
  <li>Ice Cream</li>
</ul>
```

The `<ul>` is the parent.

The `<li>` is the child.

So this code:

```
list.appendChild(item);
```

means:

```
Put item inside list.
```

16. Core Sprint 12 Creation Pattern

This is one of the most important Sprint 12 patterns:

```
const list = document.getElementById("desserts");

const item = document.createElement("li");
item.textContent = "Ice Cream";

list.appendChild(item);
```

Read it step by step:

```
1. Find the list.
2. Create a list item.
3. Put text inside the list item.
4. Add the list item to the list.
```

This is the basic DOM creation flow.

17. Adding More Than One Element

You can create and append multiple elements.

```
const list = document.getElementById("desserts");

const firstItem = document.createElement("li");
firstItem.textContent = "Ice Cream";
list.appendChild(firstItem);

const secondItem = document.createElement("li");
secondItem.textContent = "Cake";
list.appendChild(secondItem);
```

Result:

```
<ul id="desserts">
  <li>Ice Cream</li>
  <li>Cake</li>
</ul>
```

Later, you can use arrays and loops to avoid repeating this pattern manually.

Part G: Removing Elements with `removeChild()`

18. What Is `removeChild()`?

`removeChild()` removes a child element from a parent element.

Example HTML:

```
<ul id="tasks">
  <li id="task1">Study JavaScript</li>
</ul>
```

JavaScript:

```
const list = document.getElementById("tasks");
const task = document.getElementById("task1");

list.removeChild(task);
```

Result:

```
<ul id="tasks"></ul>
```

Simple meaning:

```
The parent removes the child.
```

19. Important Rule for `removeChild()`

`removeChild()` is called on the parent.

Correct:

```
list.removeChild(task);
```

This means:

```
Ask the list to remove the task inside it.
```

Incorrect beginner mental model:

```
task.removeChild();
```

That is not the right pattern for `removeChild()`.

Beginner rule:

```
removeChild() belongs to the parent element.
```

20. Removing a Created Element

You can create an element, append it, and remove it later.

```
const list = document.getElementById("tasks");

const task = document.createElement("li");
task.textContent = "Practice DOM";

list.appendChild(task);
list.removeChild(task);
```

What happens:

```
1. Create the task.
2. Add the task to the list.
3. Remove the task from the list.
```

Part H: Attributes and `setAttribute()`

21. What Is an Attribute?

An attribute gives extra information to an HTML element.

Examples:

```
<img src="cat.jpg" alt="A cat">
<button aria-expanded="false">Menu</button>
<li data-kind="cold">Ice Cream</li>
```

Attributes include:

- `id`
- `class`
- `src`
- `alt`
- `href`
- `type`
- `aria-expanded`
- `data-id`
- `data-kind`

Simple meaning:

```
Attributes describe or configure HTML elements.
```

22. What Is `setAttribute()`?

`setAttribute()` adds or changes an attribute on an element.

Syntax:

```
element.setAttribute("attributeName", "attributeValue");
```

Example:

```
const item = document.createElement("li");
item.textContent = "Ice Cream";
item.setAttribute("data-kind", "cold");
```

Result:

```
<li data-kind="cold">Ice Cream</li>
```

Simple meaning:

```
setAttribute() sets an attribute on an element.
```

23. Setting Common Attributes

Example with an image:

```
const image = document.createElement("img");
image.setAttribute("src", "cat.jpg");
image.setAttribute("alt", "A cat sitting on a chair");
```

Result:

```
<img src="cat.jpg" alt="A cat sitting on a chair">
```

Example with a button:

```
const button = document.createElement("button");
button.textContent = "Menu";
button.setAttribute("aria-expanded", "false");
```

Result:

```
<button aria-expanded="false">Menu</button>
```

Example with custom data:

```
const item = document.createElement("li");
item.textContent = "Ice Cream";
item.setAttribute("data-kind", "cold");
```

Result:

```
<li data-kind="cold">Ice Cream</li>
```

24. Why `alt` Text Matters

When creating an image, always think about `alt`.

Example:

```
const image = document.createElement("img");
image.setAttribute("src", "profile.jpg");
image.setAttribute("alt", "Profile photo of the user");
```

The `alt` attribute helps describe the image when the image cannot be seen or loaded.

Beginner rule:

```
Images should usually have useful alt text.
```

Part I: CSS Classes and `classList`

25. What Is `classList`?

`classList` lets JavaScript add, remove, or toggle CSS classes on an element.

A CSS class is usually defined in CSS.

Example CSS:

```
.highlight {
  background: yellow;
}

.hidden {
  display: none;
}
```

Then JavaScript can apply those classes.

Example:

```
const item = document.querySelector("li");
item.classList.add("highlight");
```

Simple meaning:

```
classList changes which CSS classes an element has.
```

26. `classList.add()`

`classList.add()` adds a class.

```
item.classList.add("highlight");
```

Before:

```
<li>Ice Cream</li>
```

After:

```
<li class="highlight">Ice Cream</li>
```

Simple meaning:

```
Add this class to the element.
```

27. `classList.remove()`

`classList.remove()` removes a class.

```
item.classList.remove("highlight");
```

Before:

```
<li class="highlight">Ice Cream</li>
```

After:

```
<li>Ice Cream</li>
```

Simple meaning:

```
Remove this class from the element.
```

28. `classList.toggle()`

`classList.toggle()` switches a class on or off.

```
item.classList.toggle("hidden");
```

If the element does not have the class, `toggle()` adds it.

If the element already has the class, `toggle()` removes it.

Simple meaning:

```
If class exists, remove it.
If class does not exist, add it.
```

Example:

```
<p id="message">Hello</p>
```

```
.hidden {
  display: none;
}
```

```
const message = document.getElementById("message");
message.classList.toggle("hidden");
```

The message becomes hidden.

Run the same line again:

```
message.classList.toggle("hidden");
```

The message becomes visible again.

29. Why `classList` Is Useful

`classList` is useful because it keeps styling in CSS.

Less clean:

```
message.style.color = "red";
message.style.backgroundColor = "yellow";
message.style.padding = "10px";
message.style.border = "1px solid black";
```

Cleaner:

```
message.classList.add("warning");
```

CSS:

```
.warning {
  color: red;
  background-color: yellow;
  padding: 10px;
  border: 1px solid black;
}
```

Beginner rule:

```
Use classList when changing visual state.
Keep most styling in CSS.
```

Part J: Direct Styling with `style`

30. What Is the `style` Property?

The `style` property lets JavaScript change inline CSS directly.

Example:

```
const message = document.getElementById("message");

message.style.color = "red";
message.style.fontWeight = "bold";
```

This changes the element's inline style.

Possible result:

```
<p id="message" style="color: red; font-weight: bold;">Hello</p>
```

Simple meaning:

```
style changes CSS directly from JavaScript.
```

31. JavaScript Style Property Names

CSS uses hyphenated names:

```
background-color
font-weight
```

JavaScript usually uses camelCase property names:

```
element.style.backgroundColor = "yellow";
element.style.fontWeight = "bold";
```

Examples:

CSS Property	JavaScript Style Property
`background-color`	`backgroundColor`
`font-weight`	`fontWeight`
`font-size`	`fontSize`
`border-radius`	`borderRadius`

32. When Direct Style Changes Are Fine

Direct style changes are fine for small one-off changes.

Example:

```
message.style.color = "red";
```

But direct style changes can become messy when many styles are involved.

Messier:

```
message.style.color = "red";
message.style.backgroundColor = "yellow";
message.style.padding = "10px";
message.style.border = "1px solid black";
message.style.fontWeight = "bold";
```

Cleaner:

```
message.classList.add("warning");
```

Then define styles in CSS:

```
.warning {
  color: red;
  background-color: yellow;
  padding: 10px;
  border: 1px solid black;
  font-weight: bold;
}
```

Beginner rule:

```
Use style for small direct changes.
Use classList for larger visual states.
```

Part K: Safe DOM Updates

33. Treat User Input as Untrusted

When users type into a form, JavaScript receives a string.

Example:

```
<input id="usernameInput">
<p id="output"></p>
```

```
const input = document.getElementById("usernameInput");
const output = document.getElementById("output");

output.textContent = input.value;
```

This is safer because `textContent` displays the value as text.

Avoid this with user input:

```
output.innerHTML = input.value;
```

Why?

Because `innerHTML` treats the string as HTML.

Beginner security rule:

```
User input should not be inserted with innerHTML.
```

34. Safe Way to Build Dynamic Content

Safer pattern:

```
const list = document.getElementById("comments");
const commentText = "This came from a user.";

const item = document.createElement("li");
item.textContent = commentText;
```

```
list.appendChild(item);
```

Why is this safer?

Because:

```
createElement() creates the element.
textContent adds plain text.
appendChild() places it on the page.
```

This avoids treating user input as HTML.

35. Riskier Dynamic Content Pattern

Riskier pattern:

```
const list = document.getElementById("comments");
const commentText = "This came from a user.";

list.innerHTML = `<li>${commentText}</li>`;
```

This may look convenient, but it mixes user data with HTML strings.

Beginner rule:

```
Prefer createElement() + textContent + appendChild() for user-controlled
content.
```

Part L: Full Beginner Example

36. Complete Sprint 12 Example

Create an HTML file and write this:

```
<!DOCTYPE html>
<html>
<head>
  <title>Sprint 12 Practice</title>
```

```
  <style>
    .highlight {
      background: yellow;
    }
  </style>
</head>
<body>
  <h1 id="title">Dessert List</h1>

  <ul id="desserts"></ul>

  <script>
    const title = document.getElementById("title");
    const list = document.getElementById("desserts");

    title.textContent = "My Favorite Desserts";

    const item = document.createElement("li");
    item.textContent = "Ice Cream";
    item.setAttribute("data-kind", "cold");
    item.classList.add("highlight");

    list.appendChild(item);
  </script>
</body>
</html>
```

37. Line-by-Line Explanation

```
const title = document.getElementById("title");
```

Find the heading with the id `title`.

```
const list = document.getElementById("desserts");
```

Find the list with the id `desserts`.

```
title.textContent = "My Favorite Desserts";
```

Change the heading text.

```
const item = document.createElement("li");
```

Create a new list item.

```
item.textContent = "Ice Cream";
```

Put safe plain text inside the list item.

```
item.setAttribute("data-kind", "cold");
```

Add a custom data attribute.

```
item.classList.add("highlight");
```

Add a CSS class to the item.

```
list.appendChild(item);
```

Put the new list item inside the list.

38. Final Result in the DOM

The page starts like this:

```
<h1 id="title">Dessert List</h1>
<ul id="desserts"></ul>
```

After JavaScript runs, it becomes conceptually like this:

```
<h1 id="title">My Favorite Desserts</h1>
<ul id="desserts">
  <li data-kind="cold" class="highlight">Ice Cream</li>
</ul>
```

Part M: Common Beginner Mistakes

39. Mistake: Forgetting to Select the Element First

Wrong:

```
title.textContent = "New Title";
```

Problem:

```
JavaScript does not know what title is unless you define it first.
```

Correct:

```
const title = document.getElementById("title");
title.textContent = "New Title";
```

40. Mistake: Using the Wrong ID

HTML:

```
<h1 id="mainTitle">Hello</h1>
```

JavaScript:

```
const title = document.getElementById("title");
title.textContent = "New Title";
```

Problem:

```
The HTML id is mainTitle, but JavaScript searched for title.
```

Correct:

```
const title = document.getElementById("mainTitle");
title.textContent = "New Title";
```

41. Mistake: Expecting `createElement()` to Show Something Immediately

This creates an element:

```
const item = document.createElement("li");
item.textContent = "Ice Cream";
```

But it does not show on the page yet.

You still need:

```
list.appendChild(item);
```

Correct full pattern:

```
const list = document.getElementById("desserts");

const item = document.createElement("li");
item.textContent = "Ice Cream";

list.appendChild(item);
```

42. Mistake: Calling `removeChild()` on the Child Instead of the Parent

Wrong mental model:

```
task.removeChild();
```

Correct:

```
list.removeChild(task);
```

Why?

```
The parent removes the child.
```

43. Mistake: Using `innerHTML` with User Input

Risky:

```
output.innerHTML = userInput;
```

Safer:

```
output.textContent = userInput;
```

Rule:

```
Use textContent for user input.
```

44. Mistake: Using Many Direct Style Changes Instead of a CSS Class

Less clean:

```
message.style.color = "red";
message.style.backgroundColor = "yellow";
message.style.padding = "10px";
message.style.border = "1px solid black";
```

Cleaner:

```
message.classList.add("warning");
```

CSS:

```
.warning {
  color: red;
  background-color: yellow;
  padding: 10px;
  border: 1px solid black;
}
```

Part N: Sprint 12 Core Code Patterns

45. Change Existing Text

```
<h1 id="title">Old Title</h1>
```

```
const title = document.getElementById("title");
title.textContent = "New Title";
```

46. Create and Add a New Element

```
<ul id="list"></ul>
```

```
const list = document.getElementById("list");

const item = document.createElement("li");
item.textContent = "New item";

list.appendChild(item);
```

47. Remove an Element

```
<ul id="list">
  <li id="item">Remove me</li>
</ul>
```

```
const list = document.getElementById("list");
const item = document.getElementById("item");

list.removeChild(item);
```

48. Add an Attribute

```
const item = document.createElement("li");
item.textContent = "Ice Cream";
item.setAttribute("data-kind", "cold");
```

49. Add a CSS Class

```
const item = document.querySelector("li");
item.classList.add("highlight");
```

50. Remove a CSS Class

```
item.classList.remove("highlight");
```

51. Toggle a CSS Class

```
item.classList.toggle("hidden");
```

52. Change an Inline Style

```
const message = document.getElementById("message");
message.style.color = "red";
message.style.fontWeight = "bold";
```

53. Safe User Text Pattern

```
const output = document.getElementById("output");
const userInput = "Some user text";

output.textContent = userInput;
```

54. Safer Dynamic List Pattern

```
const list = document.getElementById("comments");
const commentText = "This is a comment.";

const item = document.createElement("li");
item.textContent = commentText;

list.appendChild(item);
```

Part O: Sprint 12 Key Rules

55. Main Rules to Memorize

```
textContent = safe plain text
innerHTML = parses HTML and is risky with user input
createElement() = creates an element
appendChild() = adds a child to a parent
removeChild() = removes a child from a parent
setAttribute() = adds or changes an attribute
classList.add() = adds a CSS class
classList.remove() = removes a CSS class
classList.toggle() = switches a CSS class on or off
style = changes inline CSS directly
```

56. Safety Rules

```
Treat user input as untrusted.
Use textContent for user text.
Avoid innerHTML with user-controlled strings.
Prefer createElement() + textContent + appendChild() for dynamic content.
```

57. Style Rules

```
Use CSS for styling.
Use classList to apply CSS classes.
Use direct style changes only for small, simple changes.
Do not put many style rules directly in JavaScript if a CSS class would be
cleaner.
```

58. DOM Creation Rule

```
Creating an element is not the same as showing it.
```

You need both steps:

```
const item = document.createElement("li");
list.appendChild(item);
```

59. Parent-Child Rule

```
appendChild() and removeChild() are parent-child operations.
```

Add child:

```
parent.appendChild(child);
```

Remove child:

```
parent.removeChild(child);
```

Part P: Practice Tasks

60. Practice Task 1: Change a Heading

HTML:

```
<h1 id="title">Old Heading</h1>
```

Task:

Use JavaScript to change the heading to:

```
Sprint 12 Practice
```

Solution:

```
const title = document.getElementById("title");
title.textContent = "Sprint 12 Practice";
```

61. Practice Task 2: Add a List Item

HTML:

```
<ul id="tasks"></ul>
```

Task:

Create a new `<li>` with the text `Study DOM` and add it to the list.

Solution:

```
const tasks = document.getElementById("tasks");

const item = document.createElement("li");
item.textContent = "Study DOM";

tasks.appendChild(item);
```

62. Practice Task 3: Add an Attribute

Task:

Create an image and set its `src` and `alt` attributes.

Solution:

```
const image = document.createElement("img");
image.setAttribute("src", "profile.jpg");
image.setAttribute("alt", "Profile photo");
```

63. Practice Task 4: Add a Class

HTML:

```
<p id="message">Warning message</p>
```

CSS:

```
.warning {
  color: red;
  font-weight: bold;
}
```

JavaScript:

```
const message = document.getElementById("message");
message.classList.add("warning");
```

64. Practice Task 5: Remove an Element

HTML:

```
<ul id="tasks">
  <li id="oldTask">Old task</li>
</ul>
```

JavaScript:

```
const tasks = document.getElementById("tasks");
const oldTask = document.getElementById("oldTask");

tasks.removeChild(oldTask);
```

Part Q: Sprint 12 Minimum Standard

65. Code You Should Be Able to Explain from Memory

You are ready to move on when you can explain this code from memory:

```
const list = document.getElementById("desserts");

const item = document.createElement("li");
item.textContent = "Ice Cream";
item.setAttribute("data-kind", "cold");
item.classList.add("highlight");

list.appendChild(item);
```

Line-by-line meaning:

```
Find the list.
Create a list item.
Put safe text inside the list item.
Add an attribute to the list item.
Add a CSS class to the list item.
Put the list item inside the list.
```

66. Concepts You Must Know

You should be able to answer these:

```
What does textContent do?
What does innerHTML do?
Why is innerHTML risky with user input?
What does createElement() do?
Why is a created element not visible immediately?
What does appendChild() do?
What does removeChild() do?
Why is removeChild() called on the parent?
What does setAttribute() do?
What does classList.add() do?
What does classList.remove() do?
What does classList.toggle() do?
When should you use classList instead of style?
```

Part R: Checkpoint Questions

67. Review Questions

1. What did Sprint 11 teach that Sprint 12 builds on?
2. What is the main goal of Sprint 12?
3. What does `textContent` do?
4. Why is `textContent` safer for user input?
5. What does `innerHTML` do?
6. Why can `innerHTML` be risky?
7. What does `createElement()` create?
8. Why is a new element not visible immediately after `createElement()`?
9. What does `appendChild()` do?
10. What does `removeChild()` do?
11. Why is `removeChild()` called on the parent element?
12. What does `setAttribute()` do?
13. What is an HTML attribute?

14. What does `classList.add()` do?
15. What does `classList.remove()` do?
16. What does `classList.toggle()` do?
17. When is direct `style` editing acceptable?
18. Why is `classList` often cleaner than many direct `style` changes?
19. What is the safer pattern for displaying user-controlled text?
20. What is the core DOM creation pattern in Sprint 12?

68. Answer Key

1. Sprint 11 taught how to find/select DOM elements.
2. Sprint 12 teaches how to change page content safely and intentionally.
3. `textContent` reads or changes plain text inside an element.
4. It treats content as text, not HTML.
5. `innerHTML` reads or changes the HTML markup inside an element.
6. It can treat user-controlled strings as HTML, which can create security problems.
7. `createElement()` creates a new element in JavaScript memory.
8. Because it has not been appended to the DOM yet.
9. `appendChild()` adds a child element to a parent element.
10. `removeChild()` removes a child element from a parent element.
11. Because the parent owns the child in the DOM tree.
12. `setAttribute()` adds or changes an attribute.
13. An attribute is extra information or configuration on an HTML element.
14. `classList.add()` adds a CSS class.
15. `classList.remove()` removes a CSS class.
16. `classList.toggle()` adds the class if missing and removes it if present.
17. Direct `style` editing is acceptable for small one-off style changes.
18. `classList` keeps styles in CSS and keeps JavaScript cleaner.
19. Use `textContent`, not `innerHTML`, for user-controlled text.
20. Select parent, create child, set child text/attributes/classes, append child.

Final Sprint 12 Summary

Sprint 12 is where JavaScript starts changing the page in a real way.

The most important idea is:

```
Select an element first.
Then change it safely.
```

The most important safe content rule is:

```
Use textContent for plain text, especially user input.
Avoid innerHTML with user-controlled strings.
```

The most important creation pattern is:

```
const parent = document.getElementById("someParent");

const child = document.createElement("li");
child.textContent = "Safe text";

parent.appendChild(child);
```

The most important style rule is:

```
Use classList for visual states.
Keep most styling in CSS.
```

When you understand these patterns, you can create, edit, style, and remove page content with JavaScript.